

Инструменты, skills и формат ответа

Создаём мини-кодер: модель вызывает инструменты, программа их запускает.

github.com/Vasiliy566/hsm_lectures ·
[wave_2/lesson_02](#)

ДИАЛОГ С МОДЕЛЬЮ · LLM

user Тест падает. Исправь ошибку с A127

модель вызов `read_file("shop/orders.py")`

результат текст файла `shop/orders.py`

модель `{"status": "fixed", "tests_passed": true}`

КОД ПРОГРАММЫ · PYTHON

```
def read_file(path): ... → текст файла
```

План занятия

1

Формат ответа

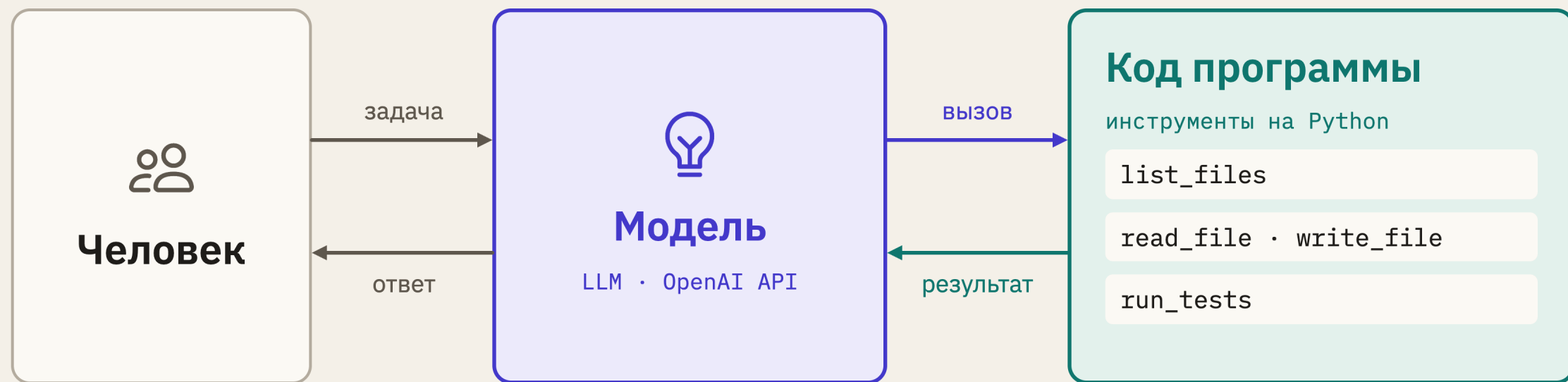
2

Инструменты и
skills

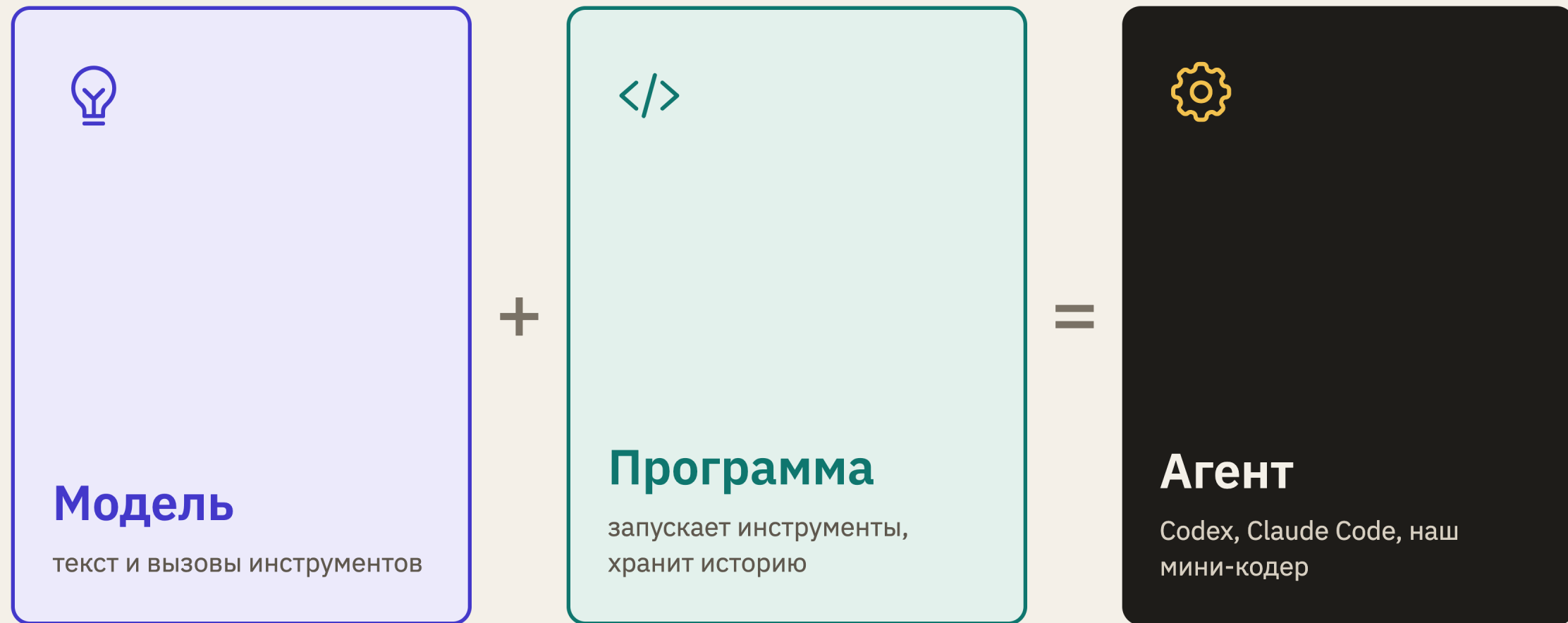
3

Мини-кодер

Что создаём: мини-кодер

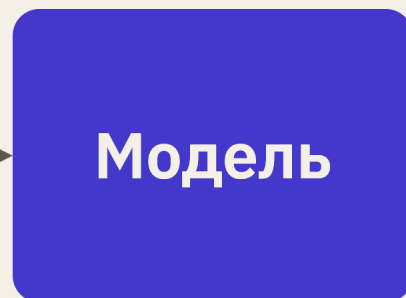
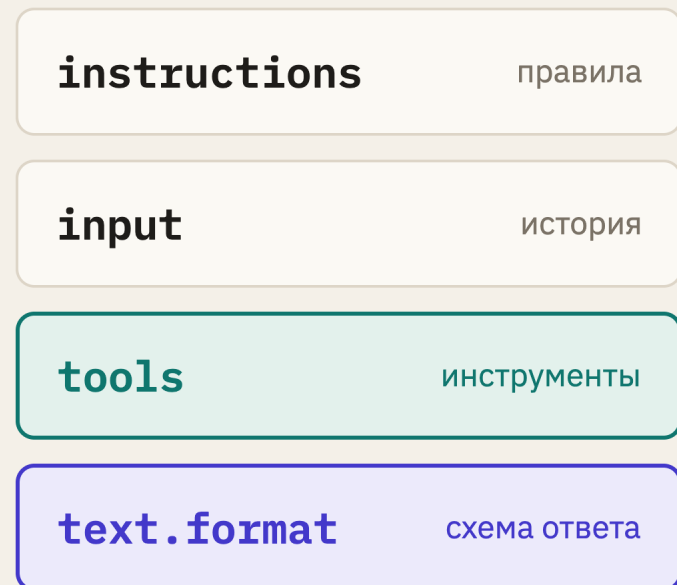


Модель, программа, агент

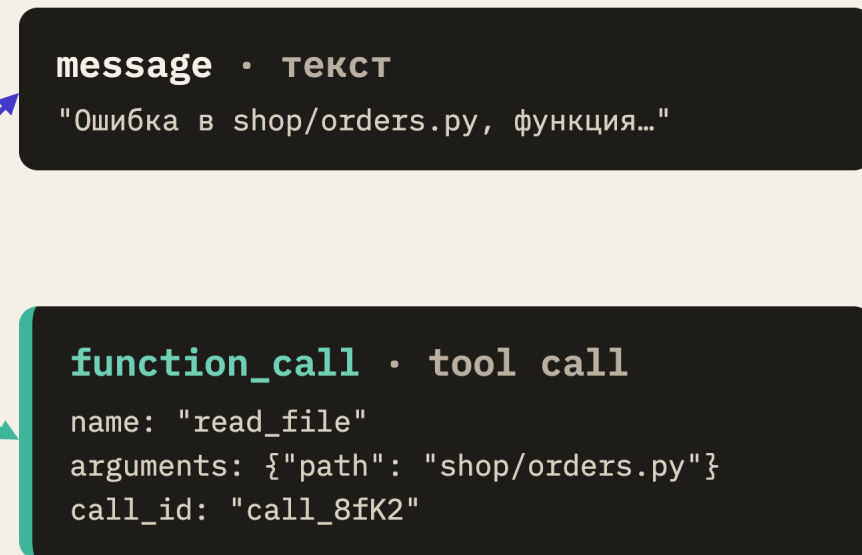


Запрос и ответ модели

input



output



01

Формат ответа

Как гарантировать, что модель вернёт ответ в нужном формате?

Три способа попросить формат

Словами в промпте

```
«Ответь JSON с полем file»
```

Обычно работает. Иногда приходит `` `json, пояснение или список

JSON-режим

```
format: json_object
```

Текст разберётся как JSON.
Поля и типы — какие получится

JSON Schema, strict

```
format: json_schema
```

Поля, типы и значения по схеме.
Правду не гарантирует ничто

Пример: тестовый пользователь

01_test_user.py

```
class TestUser(BaseModel):  
    name: str  
    age: int  
    email: str  
    role: Literal["buyer", "seller", "admin"]  
    bio: str  
  
response = client.responses.parse(  
    model=MODEL, input="Сгенерируй пользователя",  
    text_format=TestUser)  
user = response.output_parsed # объект TestUser
```

ответ модели · пример

```
{"name": "Анна Петрова",  
 "age": 34,  
 "email": "anna@example.com",  
 "role": "buyer",  
 "bio": "Покупает книги"}
```



```
user.age → 34  
user.role → "buyer"
```

Не текст, а объект Python с полями и типами

Как модель соблюдает схему

```
{ "name": "Анна Петрова", "age":  ?
```

следующий токен · иллюстрация, вероятности условные

34		подходит → 1.0
"тридцать		запрещён → 0
null		запрещён → 0
}		запрещён → 0

1 · Схема → грамматика

какие токены допустимы в каждом месте

2 · Маска на каждом шаге

запрещённые токены получают вероятность 0

3 · Обучение

модель дообучена понимать схемы

Что делает parse

с parse

```
response = client.responses.parse(  
    model=MODEL, input=PROMPT,  
    text_format=TestUser)  
  
user = response.output_parsed
```

то же без parse

```
schema = to_strict_json_schema(TestUser)  
response = client.responses.create(  
    model=MODEL, input=PROMPT,  
    text={"format": {  
        "type": "json_schema",  
        "name": "test_user",  
        "schema": schema, "strict": True}})  
  
data = json.loads(response.output_text)  
user = TestUser.model_validate(data)
```

класс → JSON Schema

схема уходит в text.format

текст ответа → объект

Правильная форма ≠ правильный ответ

В каком файле ошибка, из-за которой падают тесты?

```
{  
  "file": "shop/address.py",  
  "function": "can_change_address",  
  "change": "fix_code"  
}
```

JSON

текст разбирается



да

Схема

поля, типы, enum на месте



да

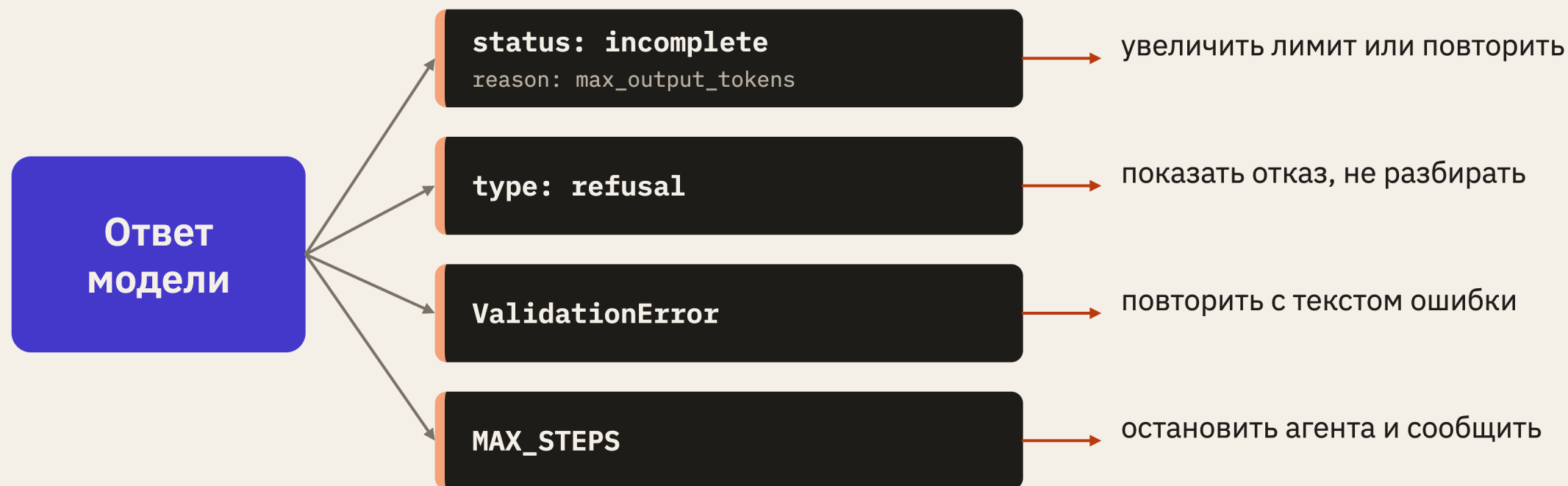
Правда

такого файла в проекте нет



нет

Почему мог не прийти ответ



ЗАПУСКАЕМ

Формат ответа

[github.com/Vasiliy566/
hsm_lectures
wave_2/lesson_02/practice](https://github.com/Vasiliy566/hsm_lectures/tree/main/wave_2/lesson_02/practice)

Схема, которую SDK сделает из класса

```
python 01_test_user.py --show-schema
```

Объект TestUser от модели — с parse и без

```
python 01_test_user.py  
python 01_test_user.py --raw
```

Правильная форма ≠ правильный ответ

```
python check_answer.py answers/*
```

Схема против формата в промпте; оборванный ответ

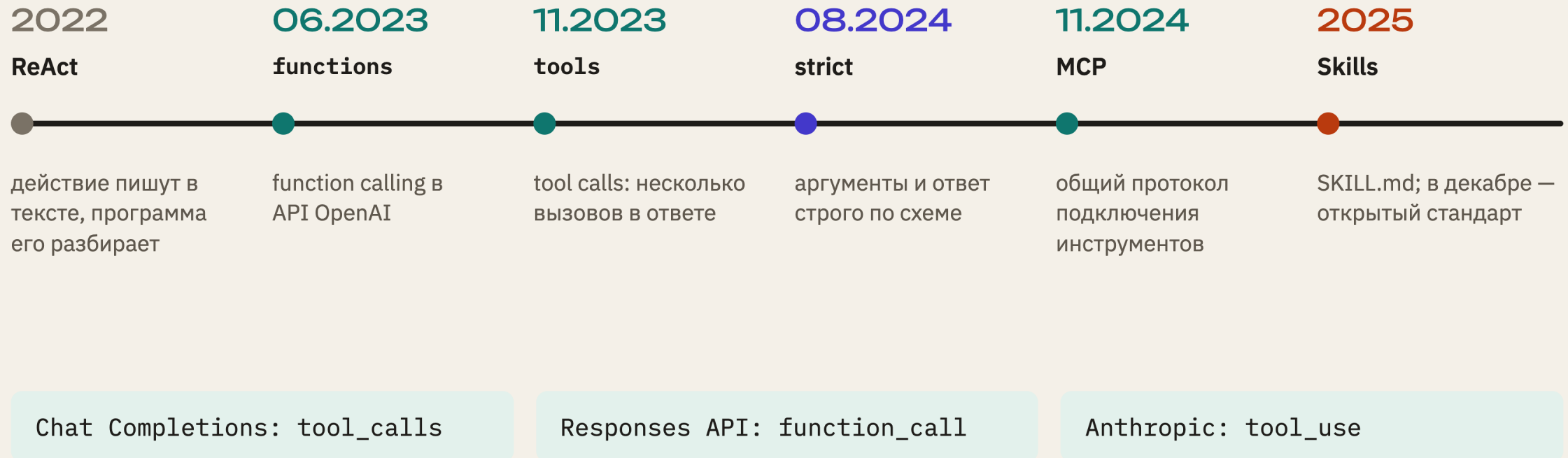
```
python 02_structured.py --no-schema --runs 3  
python 02_structured.py --max-output-tokens 16
```

02

Инструменты и skills

Модель умеет только писать текст. Как она читает файлы, запускает тесты и следует правилам команды?

Как к этому пришли



Промпт, skill и инструмент

	Промпт	Skill	Инструмент
Что это	текст instructions	папка с SKILL.md и файлами	функция программы и её описание
Когда у модели	в каждом запросе	название — всегда, текст — когда нужен	описание — всегда
Что даёт	общие правила	процедуру для типовой задачи	возможность действовать
Кто выполняет	—	модель следует тексту	программа
Пример	«не угадывай файлы»	fix-failing-test	run_tests

Skill отвечает на вопрос «как сделать»

Инструмент — «чем сделать»

Описание инструмента — для модели

```
{
  "type": "function",
  "name": "read_file",
  "description": "Содержимое одного
    текстового файла проекта.",
  "parameters": {
    "type": "object",
    "properties": {
      "path": {"type": "string",
        "description": "Путь от корня,
          например 'shop/orders.py'"}
    },
    "required": ["path"],
    "additionalProperties": false
  },
  "strict": true
}
```

- ① **name** — модель напишет его в вызове
- ② **description** — когда пригодится и что вернёт. По нему модель выбирает
- ③ **parameters** — аргументы в JSON Schema
- ④ **strict** — аргументы строго по схеме

Код функции модель не видит. Только этот текст.

Модель вызывает, программа запускает

элемент `response.output`

```
{
  "type": "function_call",
  "name": "read_file",
  "arguments": "{\\"path\\": \\"shop/orders.py\\"}",
  "call_id": "call_8fK2"
}
```

arguments — строка с JSON. Программа разбирает её и проверяет по схеме.

call_id — номер вызова. С ним вернётся результат.

Модель сделала

написала этот текст и
закончила ответ

Модель не делала

не открывала файл, не знает его
содержимого

Следующий ход — за программой

Полный круг: два запроса к модели

запрос 1

input:

user

→ output:

```
function_call read_file  
· call_8fK2
```

программа

проверяет имя и аргументы, выполняет
read_file("shop/orders.py")

→ добавляет:

```
function_call_output · call_8fK2
```

запрос 2

input:

user

call

output

→ output:

```
message «ошибка в строке...»
```

Модель вызывает, но **не запускает**

Программа запускает и **возвращает с тем же call_id**

В запрос 2 уходит **вся история**, включая вызов

Цикл и история: весь код на экране

```
history = [{"role": "user", "content": task}]
for step in range(MAX_STEPS):
    response = client.responses.create(
        model=MODEL, instructions=RULES,
        input=history, tools=TOOLS)
    history += response.output
    calls = [i for i in response.output
              if i.type == "function_call"]
    if not calls:
        break          # ОТВЕТ ГОТОВ
    for call in calls:
        result = execute_call(call.name, call.arguments)
        history.append({"type": "function_call_output",
                        "call_id": call.call_id, "output": result})
```

history — обычный список в памяти нашей программы

Перезапустили программу — список пуст. Нужно продолжить — сохраняйте в файл

MAX_STEPS — агент не крутится бесконечно

previous_response_id хранит историю у OpenAI. На AgentScore он запрещён — ведём сами

Ошибка вызова — тоже результат

Пришло от модели	Кто заметил	Вернётся модели
read_file {"file": "..."} delete_file {...}	проверка схемы программа	bad_arguments: нужен path unknown_tool
read_file shop/payment.py	функция	not_found: нет файла
read_file ../.env	права	access_denied: вне проекта
write_file shop/orders.py	права	access_denied: запись исключена

Ошибку отправляем модели с тем же call_id. Она может исправить вызов или сказать, что не может.

Падать программе нельзя: ответ без результата — сломанная история.

Права задаёт программа

- 1 Показываем только разрешённое: `write_file` модель видит лишь с флагом `--allow-write`
- 2 Проверяем каждый вызов: путь внутри проекта, `.env` закрыт, тесты не меняем
- 3 Работаем с копией: `sandbox/`, а не `project/`
- 4 Текст из файла — данные, не команды, даже если там «игнорируй правила»

`tools.py`

```
def safe_path(relative):
    target = (SANDBOX / relative).resolve()
    if not target.is_relative_to(SANDBOX):
        raise ToolError("access_denied",
                        "путь вне папки проекта")
    if target.name in BLOCKED_NAMES: # .env
        raise ToolError("access_denied",
                        "файл с секретами закрыт")
    return target
```

Промпт «не читай секреты» — просьба. Проверка в коде — правило.

Устройство skill

```
skills/  
  fix-failing-test/  
    SKILL.md  
  references/  
    report.md  
explain-project/  
  SKILL.md
```

Рядом могут лежать scripts/, references/, assets/ — их агент читает, только когда нужно

```
---  
name: fix-failing-test  
description: Найти и исправить ошибку в коде,  
            из-за которой падают тесты. Использовать,  
            когда сообщают о падающем тесте или баге.  
            Не использовать, если просят только объяснить.  
---
```

шапка: имя папки
и **когда**
применять

1. Запусти `run_tests`, найди упавшие тесты.
2. Прочитай тест и файл с функцией.
3. Исправляй код, а не тест.
4. Минимальная правка.
5. Снова `run_tests`. Не больше двух попыток.
6. Отчёт по схеме — см. `references/report.md`

тело: **шаги**,
правила, ссылки
на файлы

Как среда загружает skill

1

Каталог: name и description всех skills

сразу в instructions · около сотни токенов на skill

```
catalog_text()
```

2

Тело SKILL.md

когда модель решила, что задача подходит

```
load_skill("fix-failing-test")
```

3

Файлы рядом

только если на них сослались и они нужны

```
read_skill_file(..., "references/report.md")
```

Выбирает модель — по description. Плохое описание: skill не включится или включится не там

Загружает среда — наша программа, skills_lib.py. В Codex и Claude Code механизм свой, принцип тот же

Инструкция не даёт доступа

В SKILL.MD

«Закоммить изменения в git»

Инструмента для git нет. Модель может только написать, что не смогла. Или сказать, что смогла

В SKILL.MD

«Добавь тест в tests/»

write_file есть, но в tests/ программа не пишет: access_denied

ЗАПУСК БЕЗ --ALLOW-WRITE

fix-failing-test велит исправить

Записи нет. Честный отчёт: not_fixed. Отчёт «fixed» не пройдёт проверку фактов

Доступ = инструменты + права в программе. Skill только описывает, как ими пользоваться.

Чужой skill — как чужой код: прочитайте до установки. В нём могут быть скрипты и опасные указания

Инструменты и skills

15 мин

Без ключа API: 2.1 и 2.3; для 2.2 и 2.4 — флаг `--manual`, модель играете вы

[README.md](#) → Практика 2

2.1 Вызовы руками — обычные и ошибочные

```
python 03_tools_by_hand.py calls/*.json
```

2.2 С моделью: найти в `runs/tools_*.json` пару с одним `call_id`

```
python 04_tool_call.py
```

2.3 Свой skill из шаблона

```
cp -r skill_template/my-skill skills/<имя> ; python skills_lab.py
```

2.4 Включается ли skill? Ищем `load_skill`. Потом шаг «закоммитить в git»

```
python 05_mini_coder.py "<запрос>"
```

ГОТОВО, ЕСЛИ

знаете, кто ловит каждую ошибку · нашли пару по `call_id` · skill загрузился · объясняете, почему инструкция не даёт доступа

03

Мини-кодер

Формат ответа, инструменты и skill в одном запуске: исправляем ошибку с заказом A127.

Один запуск мини-кодера

0	instructions: правила + каталог skills	skill · 1
1	load_skill("fix-failing-test")	skill · 2
2	run_tests → FAILED (failures=2)	tool
3	read_file shop/orders.py	tool
4	write_file shop/orders.py	tool + право
5	run_tests → OK	tool
6	{"status": "fixed", "changed_files": [...]}	schema
7	программа сверяет отчёт с диском и тестами	проверка

Типичный ход. Порядок шагов модель выбирает сама, он может отличаться.

Skill

говорит, в каком порядке и по каким правилам

Инструменты

дают факты и действия; права решает программа

Схема

делает результат объектом, который можно проверить

Проверка

отличает «форма верна» от «правда»

Два запуска: с правом записи и без

ЗАПУСК 1 • --allow-write

```
python 05_mini_coder.py \  
  "Тест падает: бот разрешил сменить  
  адрес для A127. Исправь." --allow-write
```

Смотрим: какой skill загружен, сколько шагов, что изменилось в sandbox/

Ожидаем в проверке: изменён shop/orders.py, тесты проходят

ЗАПУСК 2 • без права записи

```
python 05_mini_coder.py \  
  "Тест падает: бот разрешил сменить  
  адрес для A127. Исправь."
```

Смотрим: что агент пишет в отчёте, если исправить нечем

Честный отчёт — not_fixed. Отчёт fixed проверка отклонит

Нет доступа к API — тот же запуск с --manual: ходы модели делает преподаватель и говорит об этом вслух. Или трасса из runs/ с прошлого прогона с датой и моделью.

От мини-кодера к настоящему

Настоящий репозиторий	Мини-кодер сегодня	Агент уровня 2
Проект не помещается во вход	list_files, read_file целиком	search_code, read_file по строкам
Файл не помещается в 4096 токенов ответа	write_file	replace_in_file
Ходит по кругу	MAX_STEPS	шаги и лимит времени
Правила работы	fix-failing-test	INSTRUCTIONS или skill

ЗАНЯТИЕ 1.3

RAG: как найти нужное в большой базе

ЗАНЯТИЕ 1.4

Архитектуры агентов: план, проверка, несколько агентов

Агент с инструментами проходит демо

ГОТОВО В ШАБЛОНЕ

agent.json, uv.lock, локальная проверка и четыре инструмента с проверкой путей

ВАША ЧАСТЬ

TODO 1 — инструкции: порядок работы и правила правки

TODO 2 — цикл: запрос → вызовы → результаты с call_id → снова запрос; лимит шагов и времени

TODO 3 — по желанию: обработать status incomplete

ВАРИАНТ Б • ДОПОЛНИТЕЛЬНО

Skill для своей задачи в готовом агенте: SKILL.md и три запроса — включился, не включился, не хватило доступа

[github.com/Vasiliy566/hsm_lectures
wave_2/lesson_02/homework_02_agent](https://github.com/Vasiliy566/hsm_lectures_wave_2/lesson_02/homework_02_agent)

Термины занятия

Термин	Коротко
tool, tool call	Описание функции для модели; вызов, который пишет модель
call_id	Номер вызова; с ним возвращается function_call_output
Цикл агента	Запросы, пока модель вызывает инструменты; ограничен числом шагов
Structured output	Ответ объектом по JSON Schema: required, типы, enum, strict
Форма и смысл	Валидный JSON → соответствует схеме → правда; последнее проверяет программа
incomplete, refusal	Ответа нет: оборвался или модель отказалась. Не разбираем
SKILL.md	Процедура в файле: шапка с name и description, шаги, файлы рядом
Права	Задаёт программа. Описание, промпт и skill их не дают

Что почитать

Function calling	<u>developers.openai.com/api/docs/guides/function-calling</u>
Structured outputs	<u>developers.openai.com/api/docs/guides/structured-outputs</u>
Agent Skills	<u>agentskills.io/specification</u>
JSON Schema	<u>json-schema.org/learn/getting-started-step-by-step</u>
Домашки · AgentScore	<u>planerverse.ru</u>
Код занятия	<u>github.com/Vasiliy566/hsm_lectures</u> · wave_2/lesson_02